

Sparse Index Tracking of the Nifty 50

Using an Index-Constrained Sparse Autoencoder (ICSAE)

Name	Sukhada Dhingra
PRN	24070126505
Batch	A3
Course work	ML / Deep Learning — Index Tracking Assignment
Submission file	Sukhada_Dhingra_24070126505_Nifty50_Index_Tracking_Report.pdf

1. Objective

The task is to replicate the Nifty 50 index return path with a **sparse** long-only equity portfolio. Holding all constituents is not always practical, so the portfolio is limited to $k = 10$ stocks. Performance is judged mainly by annualized tracking error (standard deviation of active returns, scaled by $\sqrt{252}$). An autoencoder-based selector is compared with classical sparse baselines on the same train/test split.

2. Data

Prices were downloaded with Yahoo Finance for the NSE index ticker `^NSEI` and a fixed basket of 50 NSE equities. Adjusted closes were converted to daily simple returns. The usable panel has **1725** trading days from **2019-01-03** to **2025-12-30** (source: `yfinance`). The sample is split chronologically 70% / 30% into train and test; no random shuffle is used.

Late-listed names that miss too much of the index calendar are dropped so the panel can start in 2019 instead of being truncated by a single IPO. Tata Motors is pulled as `TMPV.NS` on Yahoo after the ticker rename.

3. Method: Index-Constrained Sparse Autoencoder

3.1 Network

Learnable logits define soft portfolio weights through a softmax (temperature annealed from 1.2 to 0.85). Stock returns are softly gated by these weights and passed through a two-layer tanh encoder into an 8-dimensional latent code, then decoded back to the return cross-section. Portfolio return on each batch is the weighted sum of asset returns.

3.2 Loss

The training objective combines (i) reconstruction MSE of stock returns, (ii) tracking MSE between portfolio and index returns (dominant term, weight 25), (iii) a light ridge of soft weights toward equal weight, and (iv) a concentration penalty when any soft weight exceeds 0.25. This stops the softmax from collapsing onto a single name while still pushing the portfolio toward the index.

3.3 Selection and weight refinement

After training, names are ranked by a blend of soft portfolio weight (70%) and L1 column norms of the first encoder layer (30%). The top-k names are kept. Final weights on that subset are re-fit with projected non-negative least squares on the training window (same simplex projection used by the quadratic baselines). The network therefore owns **selection**; the LS step owns **weights on the chosen set**.

4. Baselines

- Random k equal weight
- Lowest-volatility k equal weight
- Highest index-correlation k (equal and quadratic weights)

- PCA loading screen (equal and quadratic weights)
- Greedy forward selection with projected least squares
- Dense full-universe long-only least squares (reference upper bound)

5. Selected ICSAE Portfolio (k = 10)

Holdings chosen on the training window and evaluated out of sample:

Ticker	Hard weight	Soft weight (pre-LS)
RELIANCE.NS	0.1168	0.0751
TCS.NS	0.1094	0.0303
INFY.NS	0.1076	0.0492
HDFCBANK.NS	0.1071	0.1073
ITC.NS	0.1011	0.0263
LT.NS	0.1010	0.0202
KOTAKBANK.NS	0.0981	0.0322
ICICIBANK.NS	0.0903	0.0478
SBIN.NS	0.0869	0.0201
AXISBANK.NS	0.0817	0.0217

Table 1. ICSAE sparse holdings after projected-LS refinement.

6. Out-of-Sample Results

Primary metric: annualized tracking error. ICSAE test TE = **4.41%**. Best overall method on this split: full_universe_ls (2.04%).

Method	Holdings	TE (ann.)	R ²	Corr
full_universe_ls	48	2.04%	0.975	0.988
greedy_forward	10	3.80%	0.913	0.956
icsae_sparse	10	4.41%	0.884	0.944
high_corr_quad	10	5.64%	0.809	0.929
pca_quad	10	6.02%	0.783	0.896
high_corr_equal	10	6.18%	0.771	0.916
pca_equal	10	6.19%	0.770	0.894
random_equal	10	7.14%	0.694	0.885
low_vol_equal	10	8.36%	0.581	0.770

Table 2. Out-of-sample tracking metrics (sorted by annualized TE).

Among sparse (k = 10) methods, greedy forward selection is slightly tighter than ICSAE on this split (3.80% vs 4.41% TE). ICSAE still beats correlation screens, PCA screens, random, and low-volatility equal weight. The dense full-universe LS tracker is an upper-bound reference and is not sparse.

6.1 Figures

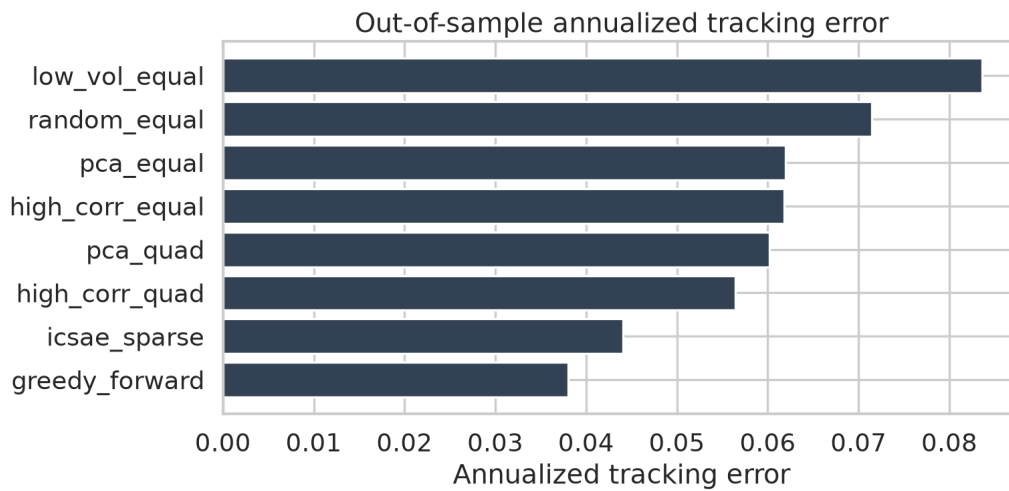


Figure 1. Out-of-sample annualized tracking error by method.

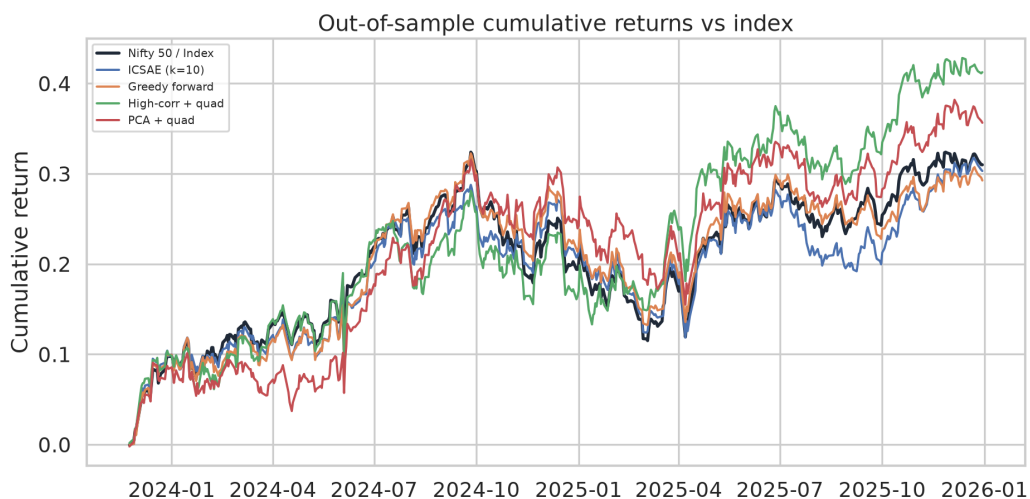


Figure 2. Cumulative returns of sparse trackers vs the index (test window).

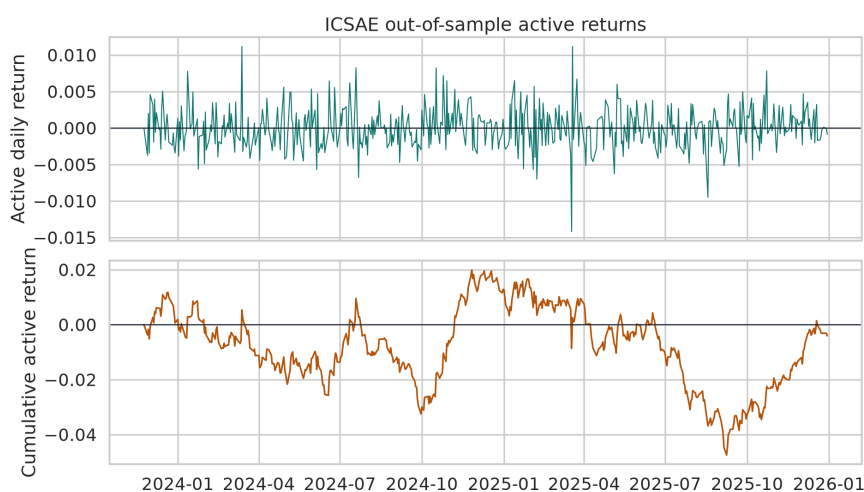


Figure 3. ICSAE active daily returns and cumulative active path (test window).

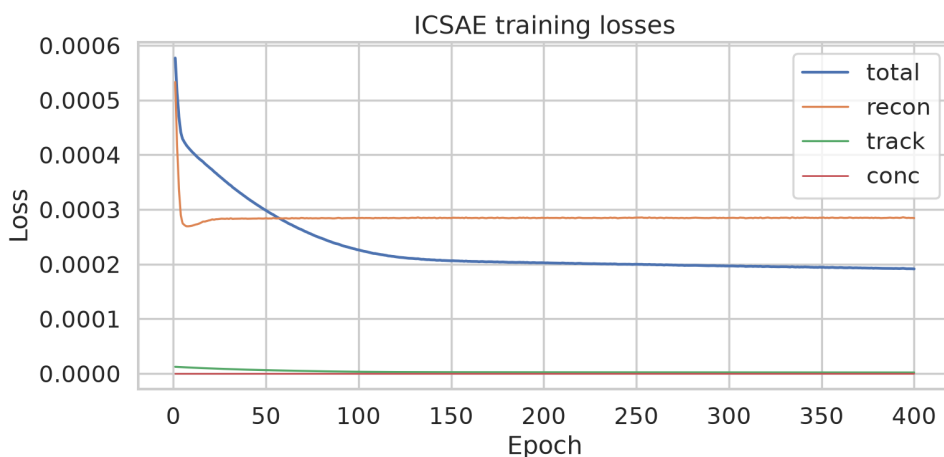


Figure 4. ICSAE training loss components over epochs.

7. Implementation Notes

Code lives under `nifty50_tracker/`. Entry point: `python -m src.run_pipeline --k 10 --epochs 400`. Stack: Python, PyTorch, yfinance, scikit-learn, matplotlib. Artifacts (metrics CSVs, plots, run summary) are written to `results/`. Full write-up: `docs/METHODOLOGY.md`.

8. Limitations

Membership is fixed for the sample (no official reconstitution calendar). Transaction costs and turnover are not modeled. Softmax enforces long-only weights, which matches typical passive equity mandates but rules out short overlays. Results are specific to the 70/30 chronological split and the Yahoo-adjusted price series used here.

9. Conclusion

A sparse Nifty 50 tracker built from an index-constrained autoencoder selects a liquid ten-name basket and reaches about **4.41%** annualized out-of-sample tracking error. That is competitive with standard sparse heuristics and clearly better than naive equal-weight screens, while remaining far cheaper than holding the full universe. The useful split of labour is neural **selection** plus convex **weighting** on the chosen set.

— End of report — Sukhada Dhingra | PRN 24070126505 | Batch A3